

VisionEngine

Voir l'interface comme un utilisateur — vision par ordinateur couplée à la vision LLM pour l'analyse et la navigation.

Résumé

VisionEngine est une boîte à outils Go découplée qui associe la vision par ordinateur classique à la vision basée sur LLM pour analyser les interfaces utilisateur, détecter les éléments d'UI et les problèmes visuels, et construire des graphes de navigation des transitions d'écrans d'applications — avec des backends de vision multi-fournisseurs enfichables et OpenCV protégé par un tag de compilation.

Description courte

Un module Go réutilisable pour l'analyse d'UI et la construction de graphes de navigation. Il propose une couche d'analyse (éléments d'UI, différences entre écrans, problèmes visuels), un graphe de navigation avec recherche de chemins en BFS et export en DOT/JSON/Mermaid, ainsi que des adaptateurs de vision LLM pour GPT-4o, Claude, Gemini, Qwen-VL et bien d'autres.

Description longue

La plupart des outils d'automatisation de tests d'UI sont en réalité aveugles. Ils s'appuient sur les arbres d'accessibilité et les sélecteurs DOM — une vision purement mécanique de l'interface — et passent à côté de tout ce qu'un humain perçoit réellement : si un bouton est effectivement affiché, si la mise en page est cassée, ou si l'écran obtenu est bien celui attendu. VisionEngine comble cette lacune en dotant l'automatisation d'une véritable perception, lui permettant d'observer une UI et d'en tirer des conclusions comme le ferait une personne. Le module s'articule autour de quatre couches coopératives, qui partent des pixels bruts pour aboutir à une compréhension globale de l'application.

La couche **Analyseur** définit un contrat stable — des interfaces (Analyzer, VideoProcessor) et des types de données (UIElement, ScreenAnalysis, ScreenDiff, Rect, Size, TextRegion, VisualIssue, ScreenIdentity, Action, KeyFrame) avec une implémentation de référence StubAnalyzer — permettant aux utilisateurs de détecter des éléments, de comparer des écrans et de mettre en évidence des problèmes visuels, le tout dans le cadre d'un contrat qui ne changera pas de manière imprévisible.

Le **Graphe de navigation** élargit la perspective d'un simple écran à l'ensemble de l'application, en la modélisant sous forme de graphe orienté des transitions d'écrans, avec une recherche de chemins en BFS et trois formats d'export (DOT, JSON, Mermaid). Ainsi, l'automatisation ne se contente pas de voir un écran : elle peut aussi planifier un itinéraire vers n'importe quel autre. Le module inclut par ailleurs des suites de tests de charge, d'automatisation, d'intégration et de sécurité pour en démontrer la fiabilité.

La couche **Vision LLM** ajoute une capacité de raisonnement multimodal moderne : une interface VisionProvider avec des adaptateurs pour OpenAI (GPT-4o), Anthropic (Claude), Gemini, Qwen-VL, Kimi, StepGUI, Astica et Ollama, organisés via une FallbackChain pour que l'échec, la limitation de débit ou la faiblesse d'un fournisseur entraîne une dégradation progressive vers le suivant, plutôt qu'un blocage complet de l'exécution.

Enfin, la couche **Configuration** gère le chargement et la validation des variables d'environnement, chaque message d'erreur destiné à l'utilisateur passant par le `il8n.Translator`.

La décision qui rend ce module réellement adoptable est que sa dépendance native lourde est optionnelle. Les bindings OpenCV sont protégés par un tag de compilation (`-tags vision`), et la version par défaut inclut des stubs — ce qui permet au module de compiler, de tester et de s'exécuter sur n'importe quel hôte Go 1.25+ sans nécessiter la chaîne d'outils OpenCV. La dépendance native n'est intégrée que si l'utilisateur l'active explicitement. C'est cette approche qui permet à VisionEngine de s'intégrer sans effort dans un runner CI standard, sans nécessiter d'image dédiée. Entièrement découplé conformément à la constitution (CONST-051(B)), il est incorporé par les consommateurs — notamment HelixQA — en tant que sous-module de codebase équivalent, offrant ainsi aux tests d'UI basés sur des preuves une véritable paire d'yeux.

Contenu

Pourquoi nous l'avons conçu

L'automatisation des tests d'interface reposant uniquement sur les arbres d'accessibilité ou les sélecteurs passe à côté de ce que l'utilisateur voit réellement. VisionEngine intègre une compréhension visuelle authentique — détection d'éléments, comparaison d'écrans et raisonnement par vision LLM — ainsi qu'une cartographie navigable des écrans d'application, permettant à l'automatisation de percevoir et de se déplacer dans une interface utilisateur.

Pourquoi c'est révolutionnaire

Cette solution réunit deux approches habituellement incompatibles — la vision par ordinateur classique, rapide et déterministe, et la vision LLM, flexible et sémantique — derrière une seule interface dotée d'une chaîne de secours. L'utilisateur bénéficie ainsi de la précision de l'une et du raisonnement de l'autre sans avoir à choisir. En rendant OpenCV strictement optionnel, elle élimine aussi le coût habituel de cette puissance : tout projet Go peut acquérir une perception réelle de l'interface sans alourdir sa chaîne de compilation avec des outils natifs de vision.

Ce qui est innovant

- **Double perception** : vision par ordinateur classique (OpenCV/GoCV) couplée à une vision LLM multi-fournisseurs avec chaîne de secours.
- **Graphe de navigation** avec recherche de chemin par parcours en largeur (BFS) et export en DOT/JSON/Mermaid.
- **Intégration conditionnelle d'OpenCV** via des balises de compilation, permettant au module de rester compilable et testable sans dépendances natives.
- **Sous-module entièrement découplé**, compatible avec l'internationalisation et partageant la même base de code (utilisé par HelixQA).

Défis et solutions

- **Friction liée aux dépendances natives lourdes** : résolue par un système de balises -tags vision et des stubs par défaut, garantissant que les environnements CI/hôtes sans OpenCV restent compilables et testables.
- **Fiabilité variable des fournisseurs de vision** : résolue par une interface VisionProvider et un compositeur FallbackChain.

- **Cartographie des flux applicatifs complexes** : résolue par un graphe de navigation orienté, complété par une recherche de chemin BFS et des exports multi-formats.
- **Couplage** : résolu via le découplage CONST-051(B) et une couche de traduction i18n.

Pile technologique (pourquoi et comment)

- **Go (1.25+)** — noyau du module et ses quatre couches.
- **GoCV / OpenCV** — vision par ordinateur classique, activée par balises de compilation.
- **Fournisseurs de vision LLM (GPT-4o, Claude, Gemini, Qwen-VL, Kimi, StepGUI, Astica, Ollama)** — raisonnement multimodal sur l'interface via des adaptateurs.
- **Algorithmes de graphes (BFS)** — recherche de chemins de navigation.
- **Exportateurs DOT / JSON / Mermaid** — visualisation du graphe de navigation.
- **Traducteur i18n** — gestion des chaînes de caractères destinées à l'utilisateur, découplée.